

Trabajo Práctico 2: Ajuste y optimización de curvas de Bézier

Métodos Computacionales, 2026

Nicolás Rodríguez López e Isaac Silva

Funciones base de Bernstein (ej 1)

Los polinomios de Bernstein son las funciones base que definen a la curva de Bézier:

$$b_0(t) = (1 - t)^3$$

$$b_1(t) = 3(1 - t)^2t,$$

$$b_2(t) = 3(1 - t)t^2$$

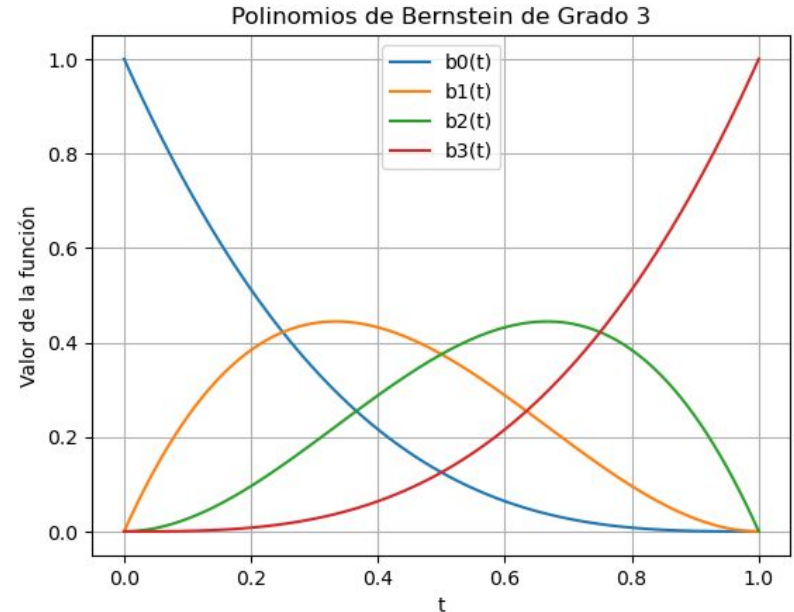
$$b_3(t) = t^3$$

Se verifica numéricamente que, para todo $t \in [0, 1]$, se cumple:

$$b_0(t) + b_1(t) + b_2(t) + b_3(t) = 1$$

Por lo tanto es una combinación lineal convexa:

$$B_3(t) = b_0(t)P_0 + b_1(t)P_1 + b_2(t)P_2 + b_3(t)P_3$$



Evaluación matricial de una curva de Bézier

(ej 2)

Construimos nuestra curva de Bezier con estos puntos

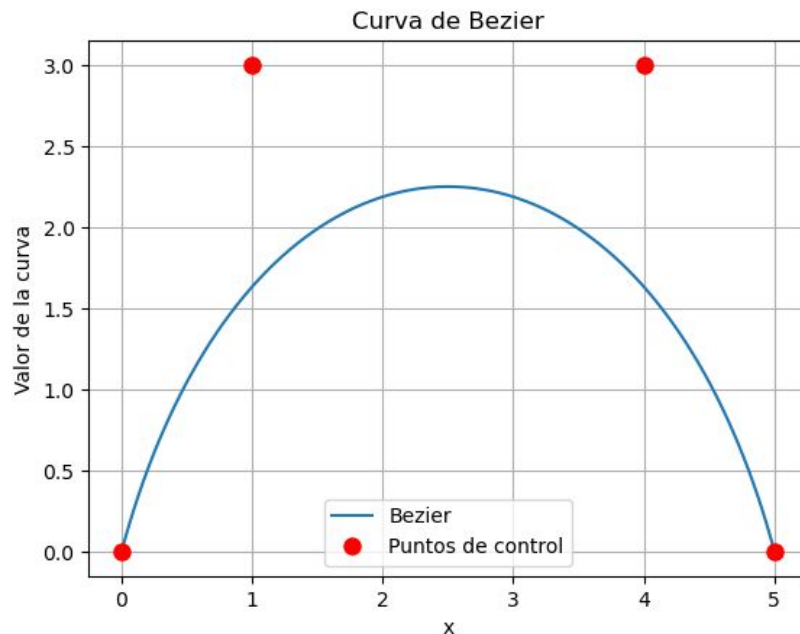
$$P_0 = (0,0), \quad P_1 = (1,3), \quad P_2 = (4,3), \quad P_3 = (5,0).$$

Usamos las siguientes matrices para hacerlo:

A: matriz que tiene los valores de los 4 polinomios de Bernstein evaluados en 100 valores de t .

P: matriz que tiene los 4 puntos de control. En la primera columna están los valores de x y en la segunda los valores de y .

C: matriz que tiene los puntos de la curva de Bézier porque multiplica los polinomios de Bernstein por los puntos de control.

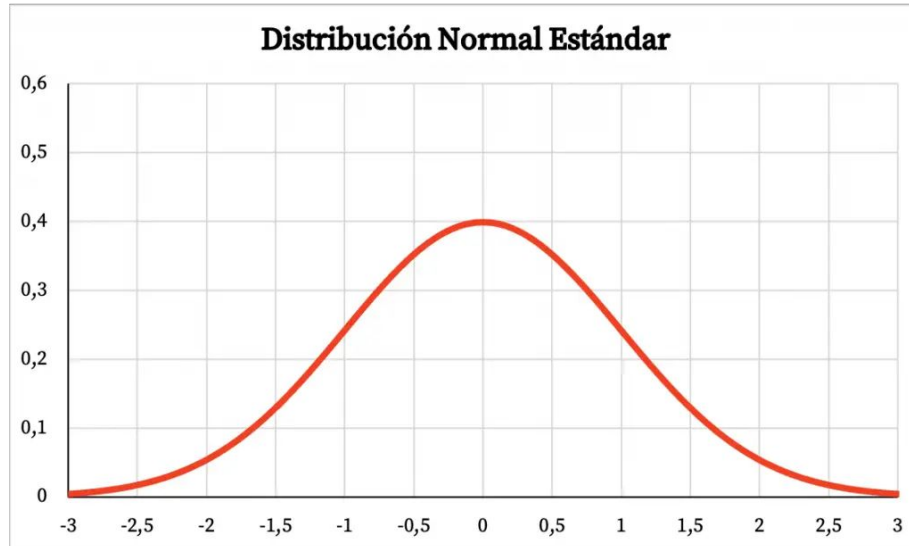


El producto **C = AP** da los puntos de la curva de Bezier

Generación de datos con ruido (ej 3)

El error sigue la distribución:

$$\eta_i \sim \mathcal{N}(0, \sigma^2)$$



$\mu = 0$: Significa que el promedio de los valores obtenidos es 0.

σ = El desvío estándar nos indica la variación/dispersión de valores respecto al centro. Mientras mayor es, es más común que nos alejemos del 0.

¿Como le agregamos ruido a una curva de Bézier?

Incisos 3) a) y b): creamos la curva $B(t)$ (la matriz C) con los puntos de control del ejercicio anterior, los “ t ” indicados en consigna y las funciones del ejercicio anterior.

Inciso c): Creamos la matriz ruido, usando la distribución pedida por la consigna. Dentro, tiene los valores aleatorios.

Matriz ruido	X	Y
$t = t_1$	0.2	-0.1
$t = \dots$	-0.13	0.03

¿Como le agregamos ruido a una curva de Bézier?

$$q_i = B(t_i) + \eta_i,$$

$$Q(t) = \text{Bézier} + \text{ruido}$$

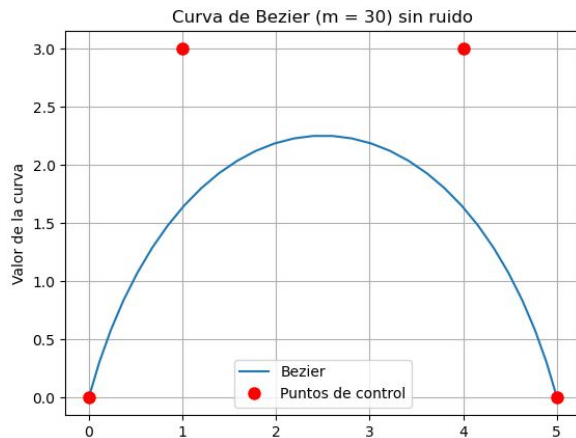
$Q(t)$	x	y
$t=t_1$	$0+0.1$	$0+0.1$
$t=...$	$0.1+0.3$	$0.1-0.1$

=

$B(t)$	x	y
$t=t_1$	0	0
$t=...$	0.1	0.1

+

Ruido	x	y
$t=t_1$	0.2	-0.1
$t=...$	-0.13	0.03



¿Como pasamos a un problema de mínimos cuadrados? (Ejercicio 4)

Lo que buscamos resolver es: Encontrar el P que minimice la distancia cuadrada entre la curva de Bézier sin ruido (AP) y la con ruido (Q)

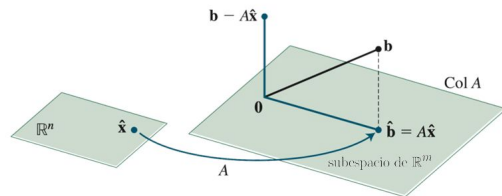
$$\hat{P} = \arg \min_{P \in \mathbb{R}^{4 \times 2}} \|AP - Q\|^2$$

Vimos que Q tiene 2 columnas, y P también tiene 2 columnas. Si llamamos q_1, q_2 a las columnas de Q , y p_1, p_2 a las de P :

$$\|AP - Q\|^2 = \|Ap_1 - q_1\|^2 + \|Ap_2 - q_2\|^2$$

En cada cada columna, tenemos un problema de mínimos cuadrados como el visto en las clases teóricas: con vectores

- Dado que $\hat{\mathbf{b}}$ es el punto de $\text{Col } A$ más cercano a $\mathbf{b}$, el vector $\hat{\mathbf{x}}$ es una solución de mínimos cuadrados de $A\mathbf{x} = \mathbf{b}$.



Como vale lo visto en la teórica, y ambas ecuaciones tienen la misma A , vale entonces usar la ecuación normal

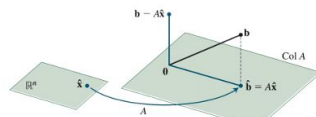
$$A^T A \hat{P} = A^T Q \quad \hat{P} = (A^T A)^{-1} A^T Q$$

$$\mathbf{b} - A\hat{\mathbf{x}} \perp \text{Col } A$$

$$A^T(\mathbf{b} - A\hat{\mathbf{x}}) = 0$$

$$A^T \mathbf{b} - A^T A \hat{\mathbf{x}} = 0$$

$$A^T A \hat{\mathbf{x}} = A^T \mathbf{b}$$



→ Ecuaciones Normales
(única o infinitas soluciones)

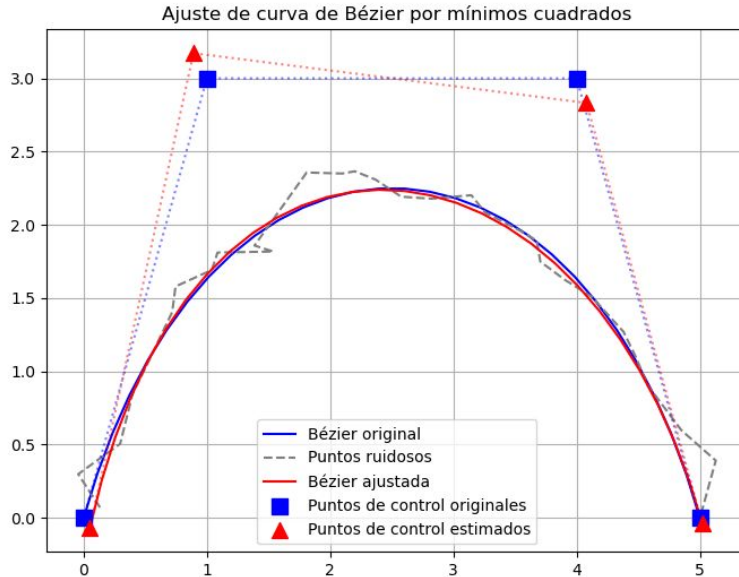
Acá está el P sombrero que minimiza la expresión $\|AP - Q\|^2$:

```
[[ 0.05064689 -0.06943931]
 [ 0.89111267  3.17213596]
 [ 4.08059359  2.83194426]
 [ 5.0192952  -0.03759889]]
```

Y acá lo resolvimos usando la forma de la ecuación normal: (nos dio lo mismo)

```
[[ 0.05064689 -0.06943931]
 [ 0.89111267  3.17213596]
 [ 4.08059359  2.83194426]
 [ 5.0192952  -0.03759889]]
```


Puntos de control originales VS Puntos de control con ruido *

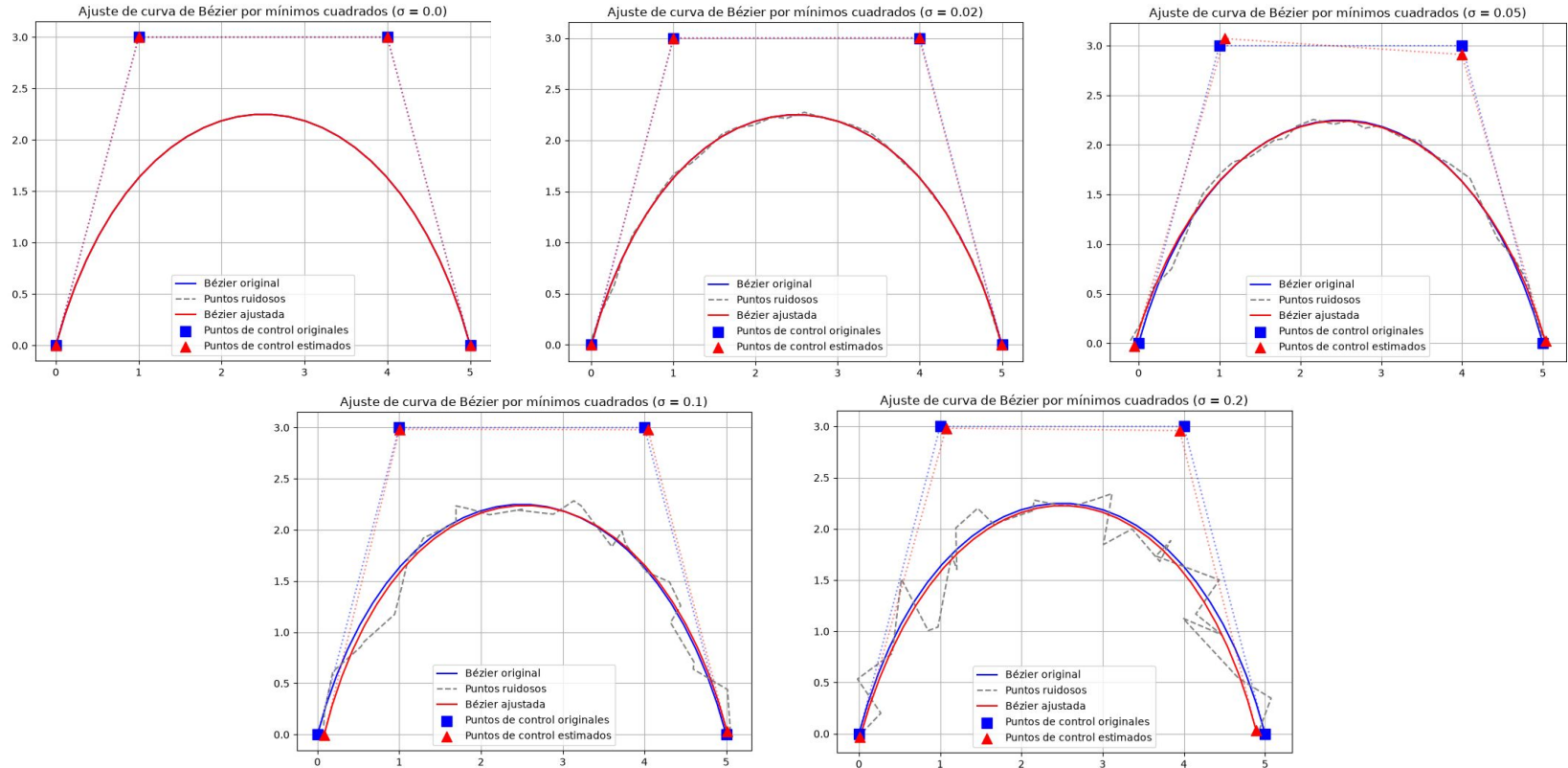


No son exactamente iguales porque la curva que tiene esos Psombrero intentará acercarse lo más posible a la curva con ruido Q (la curva con ruido), no a los puntos de la curva de bézier original AP.

Esto lo hace por la minimización de la distancia en AP-Q de la diapo anterior.

Puntos de control originales VS Puntos de control con ruido

Repetimos el procedimiento anterior para distintos niveles de ruido: (f y g)



Error cuadrático medio (h)

$$\text{ECM} = \frac{1}{m} \sum_{i=1}^m \left\| B(t_i) - \hat{B}(t_i) \right\|^2$$

Es una forma de indicarnos qué tanta distancia/diferencia hay entre la curva original y la curva ajustada.

```
sigma=0.0   ECM=0.000000  
sigma=0.02  ECM=0.000019  
sigma=0.05  ECM=0.000697  
sigma=0.1   ECM=0.001651  
sigma=0.2   ECM=0.003044
```

Podemos observar que, a medida que aumenta el sigma, tenemos mayor ECM.

Conclusiones después de ver el cambio en el sigma (aumento en el nivel de ruido) (i)

¿Qué ocurre cuando aumenta el nivel de ruido?

A medida que aumenta la desviación estándar del ruido (sigma), los puntos de datos observados (Q) se dispersan cada vez más de su ubicación verdadera (AP).

*En consecuencia, el algoritmo de mínimos cuadrados se ve forzado a promediar distancias mucho más grandes e irregulares para tratar que AP sea parecida a Q, lo que hace que el error de ajuste (y el Error Cuadrático Medio) se incremente.

¿La curva ajustada sigue recuperando razonablemente la forma original?

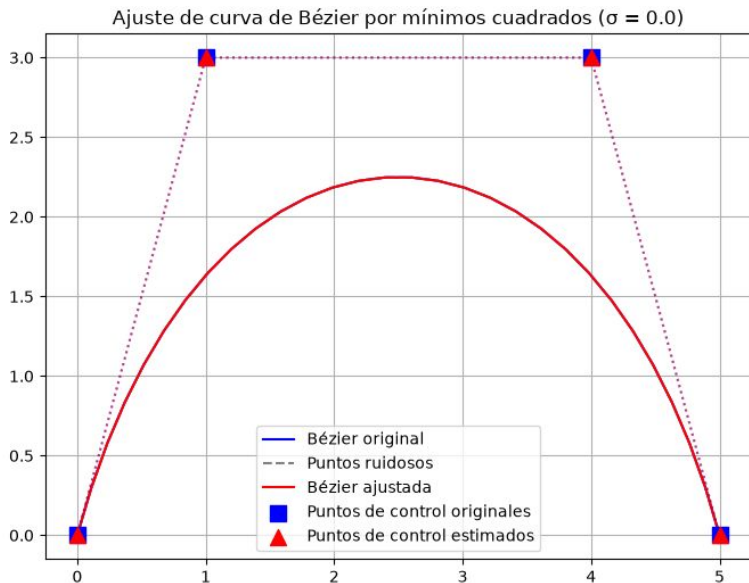
Depende del sigma. En sigmas bajos, vemos que se siguen pareciendo bastante, pero a medida que lo aumentamos, vemos cada vez menos parecida la curva ajustada a la original.

***¿Qué cambia en los puntos de control estimados? (P sombrero)**

Estos puntos son cada vez más distintos a los P originales, ya que deben “cambiarse” cada vez más para pegarse más a la la curva Q, y que así, en todos los puntos t , la distancia $AP_{\text{sombrero}}-Q$ sea menor.

Dicho de otra manera, como las puntos de control actúan como “imanes” de la curva de Bézier, el algoritmo de mínimos cuadrados desplaza estos puntos de control en direcciones erráticas para intentar acercar la curva a los puntos ruidosos.

$$\sigma \rightarrow 0$$



* En esta situación, mínimos cuadrados dice que, la expresión que minimiza la distancia $AP-Q$, son los P originales. (Recupera la curva original AP)

Esto se da porque, al ser $\sigma = 0$, no hay error que se le agregue al crear la matriz Q .

Por lo tanto, si usamos P original, $AP = Q$, lo cual minimiza la distancia (lo que busca Mínimos Cuadrados)

Vale aclarar: No existe una combinación de P sombrero que haga que $AP = Q$ en todos los t (Siempre y cuando σ sea distinto a 0).

La función de error como forma cuadrática (ej 5)

$$E(p) = \|Ap - q\|^2$$

Puede reescribirse como una forma cuadrática:

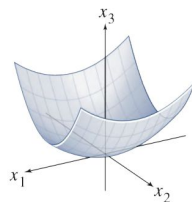
$$E(p) = \underbrace{p^\top A^\top A p}_{\text{curvatura}} \underbrace{- 2q^\top A p}_{\text{inclinación}} + \underbrace{q^\top q}_{\text{constante}}$$

Calculamos los valores propios de $A^\top A$ usando `np.linalg.eig` y nos dieron **todos positivos**

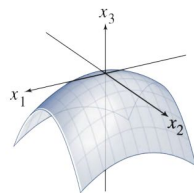
Recordando Formas cuadráticas: *

Clasificación de las formas cuadráticas

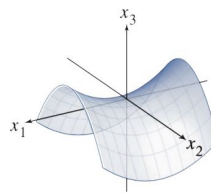
- Sea A una matriz simétrica de $n \times n$ asociada a la forma cuadrática $Q(\mathbf{x})$:
 - $Q(\mathbf{x})$ es definida positiva si y sólo si los autovalores de A **son todos positivos**.
 - $Q(\mathbf{x})$ es definida negativa si y sólo si los autovalores de A **son todos negativos**.
- **Definida positiva** si $Q(\mathbf{x}) > 0$ para todo $\mathbf{x}$ distinto de 0



Definida positiva



Definida negativa



Indefinida

Ecuaciones normales a partir del gradiente (ej 6)

$$E(p) = \|Ap - q\|^2,$$

$$\nabla E(p) = 2A^T(Ap - q).$$

Imponer $\nabla E(p) = 0$ conduce a las ecuaciones normales:

$$A^T Ap = A^T q$$

Si el gradiente de una función en un punto es igual a cero (el vector nulo), significa que estás en el **punto mínimo** de la función $E(p)$ (en el 5 supimos que tenía mínimo)

Gradiente descendiente para ajustar una curva (ej 7)

Algoritmo de Gradiente Descendiente

1. Elegir un punto inicial $\mathbf{x}_0$
2. Mientras no haya convergencia (o no se haya alcanzado un límite de iteraciones):
 - a. Computar el gradiente en el punto $\nabla f(\mathbf{x}_t)$
 - b. Computar el nuevo punto $\mathbf{x}_{t+1} = \mathbf{x}_t - \alpha \nabla f(\mathbf{x}_t)$
 - c. Verificar convergencia y actualizar $\mathbf{x}_t = \mathbf{x}_{t+1}$

¿Qué buscamos hacer en el ejercicio 7?

$$\text{minimizar } E(p) = \|Ap - q\|^2$$

¿Que es “p”?: Es un vector con las coordenadas de solo una dimensión (x o y) de puntos de control

px	x
P0x	1.2
P1x	2.3
P2x	-3.0
P3x	-2

¿Que es “E(p)”?: Es la función de error de mínimos cuadrados pero solo analizada en una dimensión (e o y).

Para cada “p”, devuelve un valor de error. Indica qué tan lejos está Ap de q .

Buscamos el valor de p que minimiza dicho error.

a) Uno de los vectores pedidos podría verse así

Implementación

Utilizamos la función gradiente del ejercicio anterior.

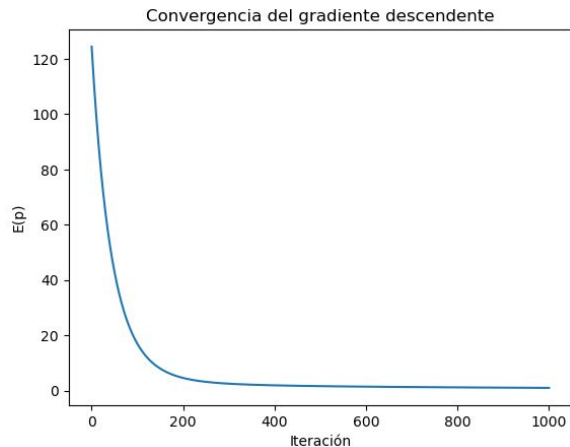
- **Alpha** = 0.001
- **Máximas iteraciones** = 1000

Existe una **tolerancia** de 0.0001 en nuestro algoritmo pensado por si llega a un mínimo evidente.

Los parámetros fueron elegidos después de algunas pocas pruebas con el algoritmo hasta llegar a algo que corriera en un tiempo coherente. Sabíamos que llegaríamos a un mínimo parecido al de **Mínimos cuadrados** (más detalle sig. diapo).

$$p_{t+1} = p_t - \alpha 2A^T(Ap_t - q)$$

d) Solo en x:



*¿Por qué se resta el gradiente?

El gradiente siempre apunta hacia la dirección donde el error *sube* más rápido. Como nuestro objetivo es minimizar (bajar el error), multiplicamos el gradiente por un signo negativo para dar un paso en la dirección contraria,

Resultados y comparaciones con Mínimos Cuadrados

*¿Por qué nos quedó distinto a mínimos cuadrados?

Seguramente, por un tema de la elección de α , ya que es bastante chico. (En el ejercicio 8 veremos si al cambiar α llegamos a un P más cercano al de mínimos cuadrados)

*¿Por qué se hace esto si existe Mínimos Cuadrados?

Mínimos Cuadrados pide invertir matrices, lo cual suele ser costoso.

Además, existen problemas que no es viable usar mínimos cuadrados y si es viable usar algoritmos de minimización para acercarse (podadora)

```
[-0.26325184  2.01306957  2.90689625  5.38051177]
```

Veamos si es parecido al mínimo encontrado por Mínimos cuadrados en x

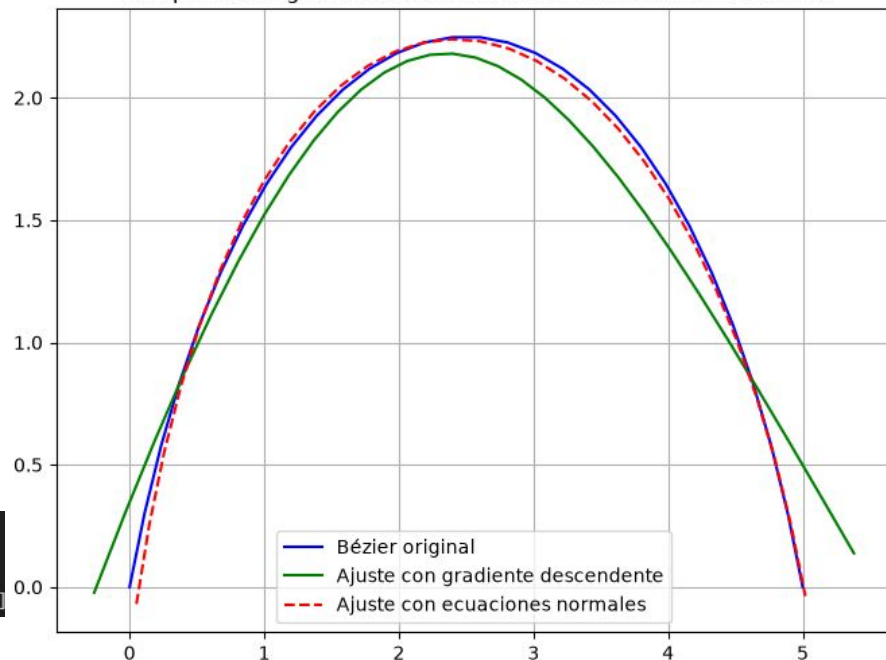
```
Este era el P_sombrero con la ecuación normal en  $x$ : [0.05064689  0.89111267  4.08059359  5.0192952 ]
```

```
El mínimo local ocurre en  $p = [ 0.25852724  2.25605575  3.55978548 -0.1934139 ]$  y  $f(x)$  vale: 0.7562233646468854
```

Veamos si es parecido al mínimo encontrado por Mínimos cuadrados en y

```
Este era el P sombrero con la ecuación normal en  $y$ : [-0.06943931  3.17213596  2.83194426 -0.03759889]
```

Comparación: gradiente descendente vs ecuaciones normales



Cambiando alpha (Ej 8)

Converger lentamente

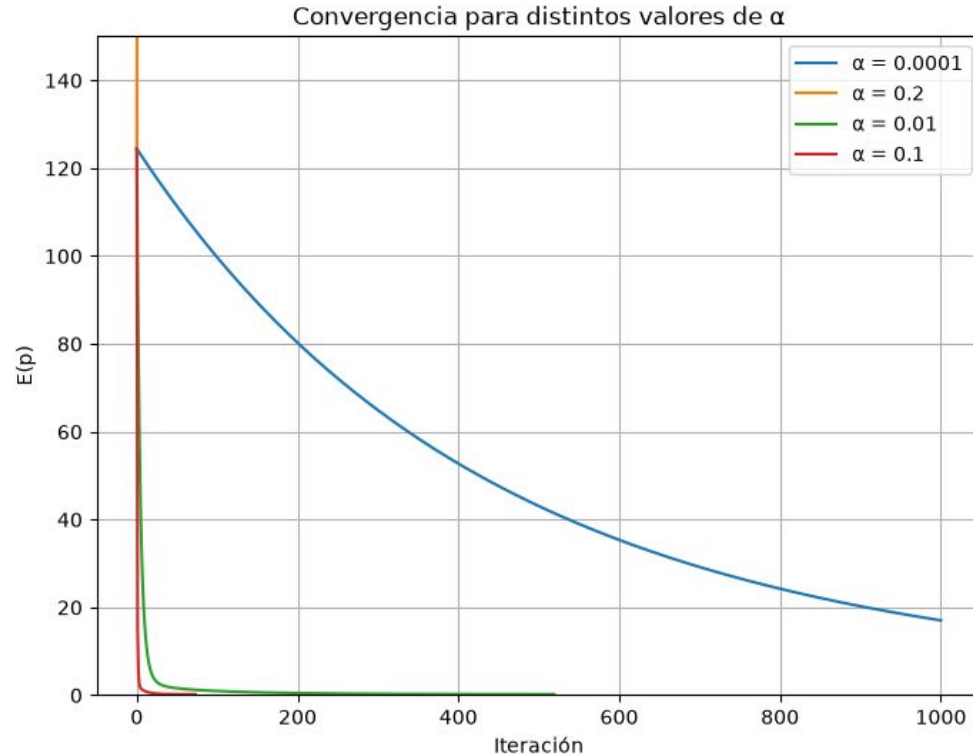
Alpha = 0.0001 no llega ni a converger a un error cercano a 0 a pesar de todas las iteraciones puestas.

Caso donde diverge

Alpha = 0.2 se va del gráfico.

Buena convergencia

Alpha = 0.1 y 0.01 convergen cerca de 0 en una cantidad de iteraciones razonables.



Conclusiones sobre el rol de alpha

- **Alpha** es (como en la analogía vista en la teórica) el “tamaño del paso” en cada iteración del algoritmo.
- Mientras que el gradiente negativo nos indica la *dirección* correcta hacia donde el error disminuye, alpha multiplica a ese vector para determinar *qué tan lejos* avanzamos en esa dirección antes de volver a evaluar si mejoramos (es menor) o no.
- Conocemos los siguientes casos de elección de alpha:

alpha muy chico (Convergencia lenta): El algoritmo es extremadamente seguro y siempre baja por la pendiente, pero da pasos microscópicos. Se requieren muchísimas iteraciones (y mayor tiempo de cómputo) para llegar al error mínimo, tal como se observó con $\alpha = 0.0001$.

alpha adecuado (Convergencia óptima): El paso es lo suficientemente grande como para llegar al mínimo de la función de error de manera rápida y eficiente (en pocas iteraciones), pero lo suficientemente moderado para no pasarse de largo.

alpha grande (Oscilación): El algoritmo da pasos tan largos que “salta” por encima del punto mínimo, cayendo en la pared opuesta de la función de error. Empieza a rebotar de un lado al otro del pozo, tardando en estabilizarse.

alpha demasiado grande (Divergencia): Es el peor escenario. El paso es tan excesivo que el rebote de un lado al otro es cada vez más alto. En lugar de descender hacia el mínimo, el algoritmo “trepa” por las paredes de la función cuadrática y el error explota hacia el infinito, como ocurrió en la prueba con $\alpha = 0.2$

Método de Newton para el mismo problema (ej 9)

Para la función cuadrática

$$E(p) = \|Ap - q\|^2$$

el Hessiano es

$$H = 2A^T A$$

Implementamos el método de Newton usando la actualización iniciando desde un vector aleatorio p_0 .

$$p_{t+1} = p_t - H^{-1} \nabla E(p_t)$$

Y en una sola iteración encontró el mínimo y era igual al conseguido en mínimos cuadrados!

```
El mínimo encontrado al final entonces es:
```

```
[0.05064689 0.89111267 4.08059359 5.0192952 ]
```

```
Mi mínimo encontrado por mínimos cuadrados para la coordenada x era: [0.05064689 0.89111267 4.08059359 5.0192952 ]
```

¿Por qué pasa esto?

El método de Newton funciona aproximando f cerca de un punto p_t por su expansión de Taylor de segundo orden:

$$f(x_{t+1}) \approx f(x_t) + f'(x_t)(x_{t+1} - x_t) + f''(x_t) \frac{(x_{t+1} - x_t)^2}{2}$$

y minimizando esa aproximación en vez de f directamente. La actualización:

$$x_{t+1} = x_t - \nabla^2 f(x_t)^{-1} \nabla f(x_t)$$

sale de poner a cero el gradiente de esa aproximación cuadrática.

Lo que pasó: Como $f(E(p))$ en este caso) ya es exactamente cuadrática, esa "aproximación" de Taylor no es una aproximación, es la **función misma**. De ese modo, Newton no da un paso aproximado sino que da el paso exacto al mínimo.

Ajuste con extremos fijos (ej 10)

En este ejercicio tuvimos que imponer

$$P_0 = q_1, \quad P_3 = q_m$$

Y pasar los términos conocidos al lado derecho para obtener un problema de mínimos cuadrados sólo en P1 y P2.

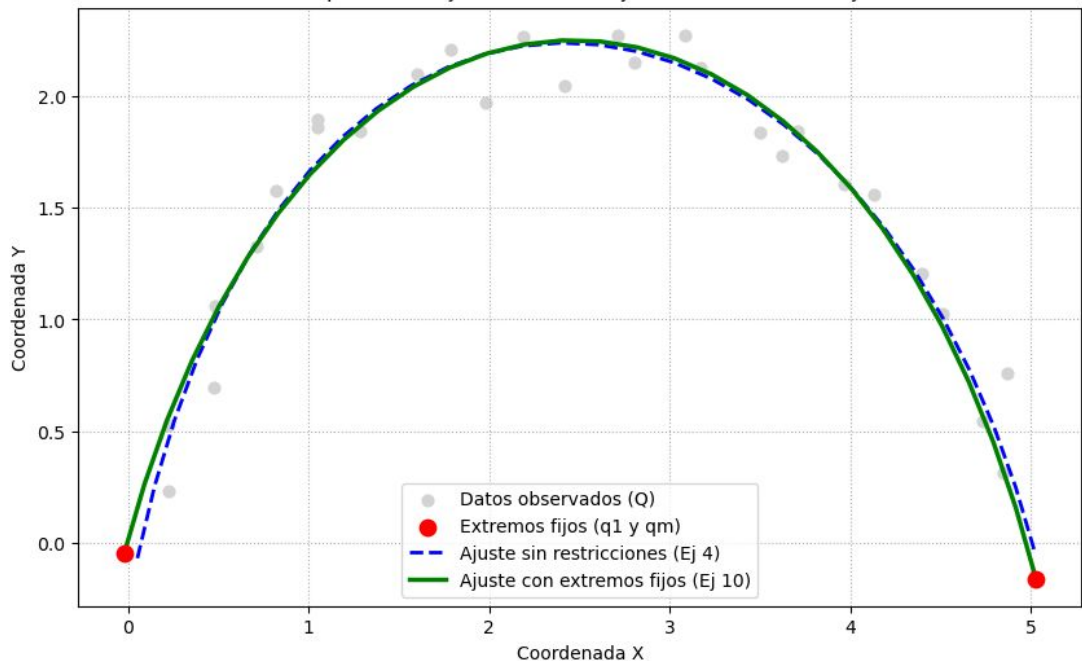
$$b_0(t_i)P_0 + b_1(t_i)P_1 + b_2(t_i)P_2 + b_3(t_i)P_3 = q_i$$

$$b_1(t_i)P_1 + b_2(t_i)P_2 = q_i - b_0(t_i)P_0 - b_3(t_i)P_3$$

```
Dimensión de la nueva matriz A_int: (30, 2)
Los puntos intermedios estimados (P1 y P2) son:
[[0.99430959  3.06449433]
 [4.0211699  3.00937258]]

La matriz de control final (P_sombrero con extremos fijos) es:
[[-0.02321823 -0.05017289]
 [ 0.99430959  3.06449433]
 [ 4.0211699  3.00937258]
 [ 5.02730493 -0.16185707]]
```

Comparación: Ajuste Libre vs. Ajuste con Extremos Fijos



- El ajuste con extremos fijos (**verde**) pasa exactamente por q_1 y q_m porque esas coordenadas se imponen como restricción y no se estiman.
- El ajuste libre (**azul**) minimiza el error total sobre los 4 puntos de control sin esa restricción, por lo que la curva resultante puede no tocar exactamente los extremos observados (que también tienen ruido).

Como consecuencia, fijar los extremos reduce el error en esos dos puntos a cero, pero en general aumenta (o deja igual) el error cuadrático total respecto del ajuste libre, ya que se está optimizando sobre un espacio más restringido (solo P_1 y P_2 , en vez de los 4 puntos de control).

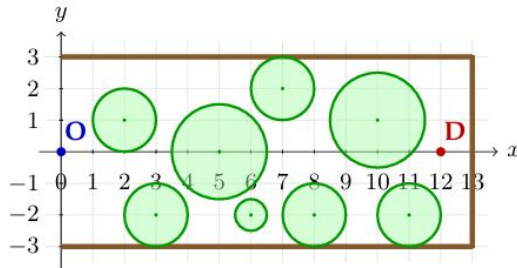
Aplicación: una trayectoria para la podadora robot (ej 11)

Una podadora robot debe desplazarse desde un punto de origen $O = (0, 0)$ hasta un punto destino $D = (12, 0)$ dentro de un jardín. El jardín tiene paredes y varios arbustos circulares que la podadora debe evitar.

Los extremos estarán siempre fijos:

$$P_0 = O = (0, 0), \quad P_n = D = (12, 0),$$

Los puntos intermedios $P_1, \dots, P_{n-1}$ serán las *variables a optimizar* (inciso 12)



j	c_j	r_j
1	(2, 1)	1
2	(3, -2)	1
3	(5, 0)	1,5
4	(7, 2)	1
5	(8, -2)	1
6	(6, -2)	0,5
7	(10, 1)	1,5
8	(11, -2)	1

¿Cómo creamos curvas de Bézier de grado arbitrario?

$$B_n(t) = \sum_{k=0}^n b_{k,n}(t) P_k, \quad b_{k,n}(t) = \binom{n}{k} (1-t)^{n-k} t^k, \quad t \in [0, 1].$$

*dado un “n” hay n+1 términos en la Bézier. Y por lo tanto, n+1 puntos de control.

- a) Función para obtener la matriz “A” (polinomios de bernstein) analizada en todos los valores de “t” que le son pasados, con un “n” (grado) pasado por parámetro. *Utiliza las fórmulas de polinomios de bernstein de grado arbitrario para crearla.*
- b) Función que hace el producto AP (matriz que representa a la curva de Bézier B(t)), luego de usar la del inciso a) para obtener A con “n” arbitrario.

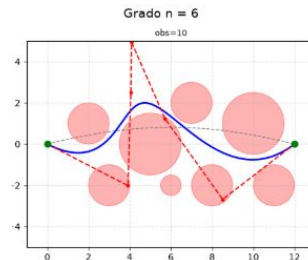
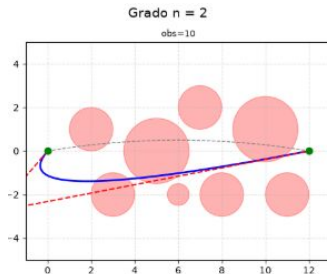
¿Cuándo hay colisión en el modelo?

$$\min_i \|B_n(t_i) - c_j\| < r_j.$$

c) La norma de una resta nos da la distancia. Con el $\min\{\}$ obtenemos el que hace más chica dicha distancia entre algún punto de la Bézier y el centro de algún obstáculo j . Si eso se cumple, en la función ponemos un `True`, indicando que hubo colisión.

Problemas de grados bajos y altos (inciso g)

- Una curva de Bézier de grado n tiene $n+1$ puntos de control y, en general, puede generar a lo sumo $n-1$ "cambios de curvatura" o inflexiones.
- Un **grado bajo** significa tener muy pocos puntos de control intermedios (con $n=2$ solo hay un punto P_1 móvil). Esto le quita "articulaciones" a la curva, haciéndola rígida e incapaz de serpentear entre múltiples obstáculos.
- Un **grado excesivamente alto** introduce "oscilaciones" o trayectorias innecesariamente complejas. La curva tiene tantos puntos de control que empieza a dar vueltas erráticas o a alejarse demasiado de un camino directo y suave.



Optimización automática del grado y de la trayectoria (ej 12)

La función de costo tendrá tres términos:

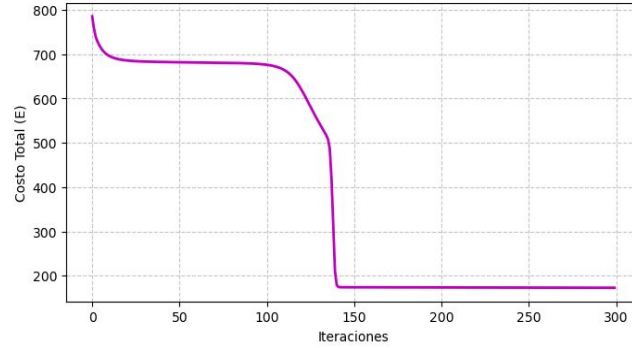
$$E(\theta) = E_{\text{largo}}(\theta) + \lambda_{\text{suavidad}} E_{\text{suavidad}}(\theta) + \lambda_{\text{obs}} E_{\text{obstáculos}}(\theta).$$

Estos son los términos de la función:

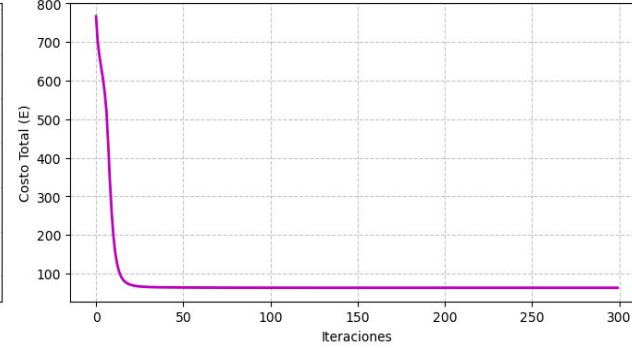
$$E_{\text{largo}}(\theta) = \sum_{i=1}^{N-1} \|B_n(t_{i+1}) - B_n(t_i)\|^2 \quad E_{\text{obstáculos}}(\theta) = \sum_{i=1}^N \sum_{j=1}^M [\text{máx}(0, r_j + \delta - \|B_n(t_i) - c_j\|)]^2$$

$$E_{\text{suavidad}}(\theta) = \sum_{k=1}^{n-1} \|P_{k-1} - 2P_k + P_{k+1}\|^2$$

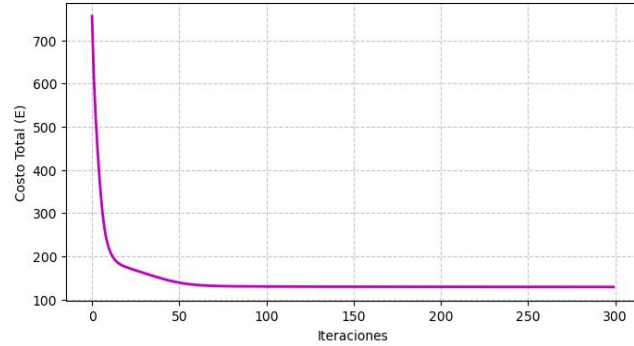
Evolución del Costo E - Grado 2



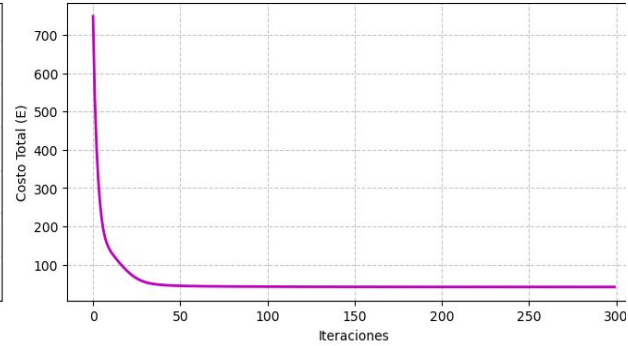
Evolución del Costo E - Grado 3



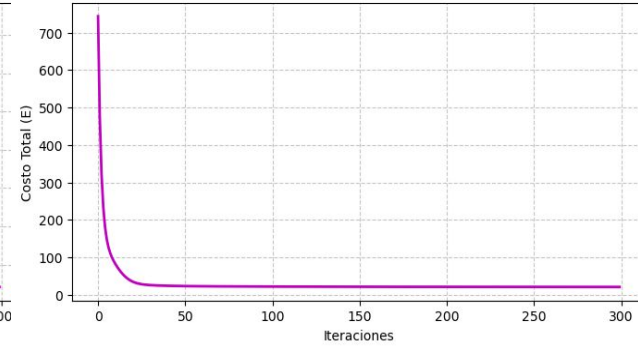
Evolución del Costo E - Grado 4

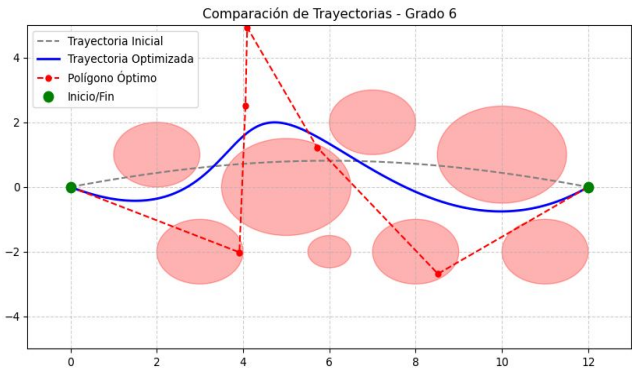
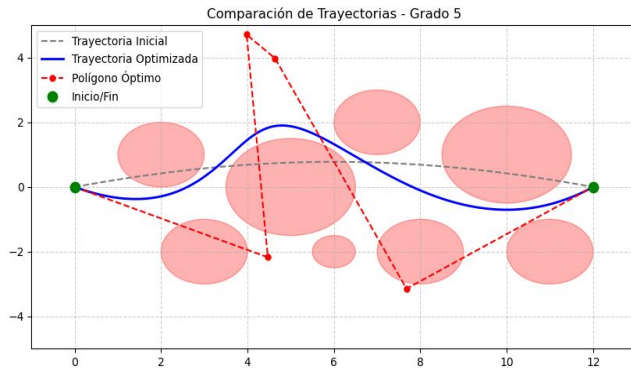
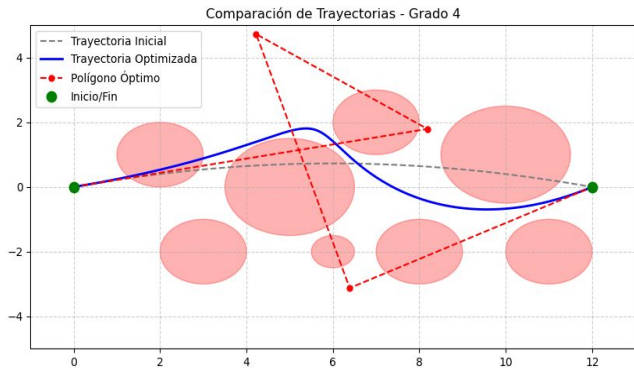
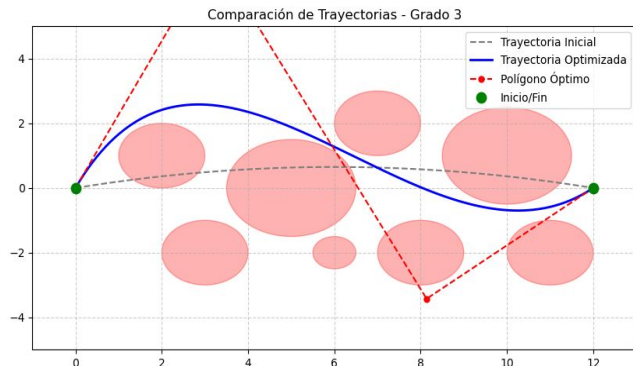
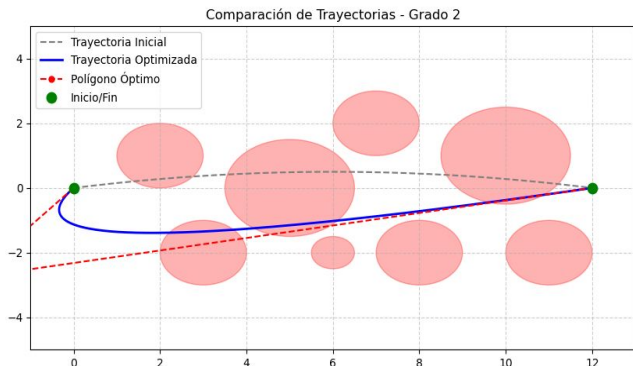


Evolución del Costo E - Grado 5



Evolución del Costo E - Grado 6





Nuestro Criterio de Factibilidad: Flexibilidad Operativa

Para determinar si una trayectoria optimizada es factible (válida y útil para la podadora), utilizaremos un criterio realista que busca un equilibrio entre seguridad y eficiencia. Exigir un respeto absoluto y milimétrico del margen de seguridad ò muchas veces obliga al algoritmo a generar maniobras erráticas o rodeos inmensos que arruinan la lógica del recorrido. Por lo tanto, nuestras condiciones de factibilidad son:

1. **Tolerancia en el margen de seguridad ($E_{\text{obstáculos}} < 1$):** es preferible "rozar" levemente la zona de advertencia del arbusto (sin llegar a chocarlo físicamente) antes que obligar a la podadora a dar vueltas ineficientes por el jardín. Aceptamos una penalización mínima en este término, asumiendo el riesgo controlado.
2. **Eficiencia del recorrido ($E_{\text{total}} \leq 50$):** la trayectoria global debe mantenerse dentro de un límite de costo total razonable, evitando caminos excesivamente largos o con curvas bruscas e innecesarias.

Elección del Grado Óptimo

Aplicando nuestro criterio de flexibilidad operativa, analizamos los valores finales de las trayectorias optimizadas para cada grado n :

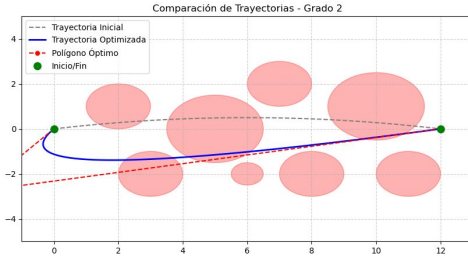
- Grado 2: $E_{\text{obs}} = [14.02]$, $E_{\text{total}} = [172.6044]$ → [No Factible]
- Grado 3: $E_{\text{obs}} = [0.96]$, $E_{\text{total}} = [63.06]$ → [No Factible]
- Grado 4: $E_{\text{obs}} = [8.42]$, $E_{\text{total}} = [128.86]$ → [No Factible]
- **Grado 5:** $E_{\text{obs}} = [0.89]$, $E_{\text{total}} = [41.95]$ → [Factible]
- Grado 6: $E_{\text{obs}} = [0.52]$, $E_{\text{total}} = [21.11]$ → [Factible]

Conclusión: nosotros observábamos que exigir un error de obstáculos exactamente igual a cero **descartaba curvas que visualmente eran muy buenas**. Al flexibilizar el margen ($E_{\text{obs}} < 1$), evitamos penalizar en exceso trayectorias que son altamente eficientes en distancia y suavidad.

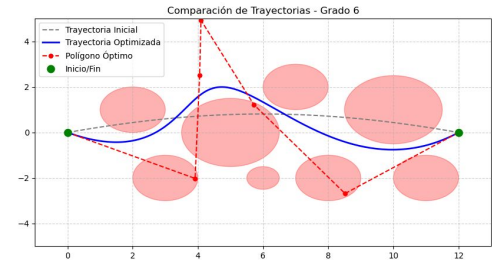
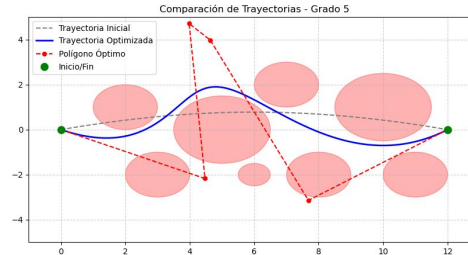
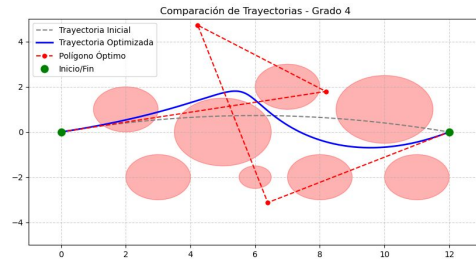
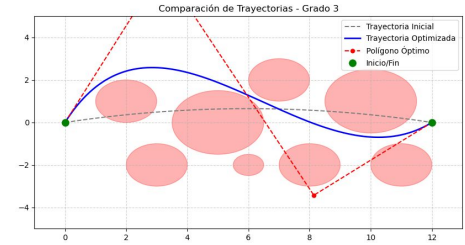
Por lo tanto, el grado elegido como la trayectoria óptima definitiva es $n = [5]$, ya que representa el grado más bajo que logra cruzar el jardín eficientemente, asumiendo un roce aceptable en los márgenes pero garantizando que no hay colisiones reales.

Análisis de las trayectorias obtenidas (ej 13)

Tabla de características de las curvas según el grado
(curvas del ejercicio 12)

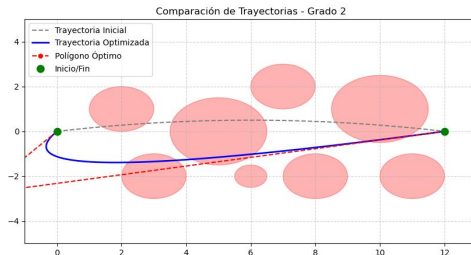


Grado	Longitud	Colisiones	Dist. mín	Costo final
2	13.5027	3	0.6536	172.5953
3	14.2131	0	1.1549	63.0630
4	13.3270	1	0.4478	128.8603
5	13.7931	0	1.1433	41.9569
6	14.0376	0	1.1446	21.1088

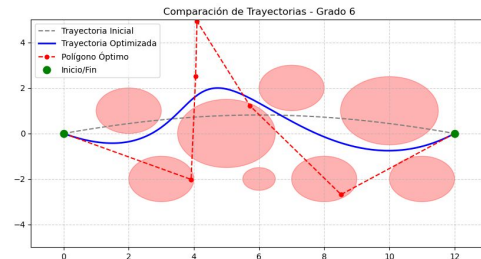
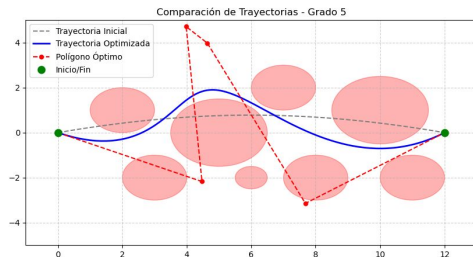


¿Que podemos decir del cambio de grado?

Problemas de tener un grado bajo: Es difícil realizar curvaturas o esquivar obstáculos con un grado bajo, por lo que suelen ser curvas que no llegan a ser factibles (Ej: grado 2)



Problemas de tener un grado alto: Mientras más grados tenemos, más curvaturas o zig-zags podemos realizar. El problema viene cuando ya no son necesarios tantas curvaturas y agregar puntos solo complejiza innecesariamente la solución (EJ: grado 5 y 6 son prácticamente iguales, pero la 6 es más compleja).

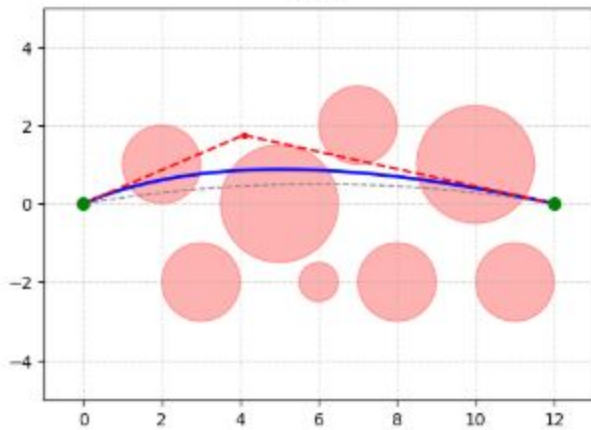


Pasamos a ver todos los gráficos con distintos lambdas

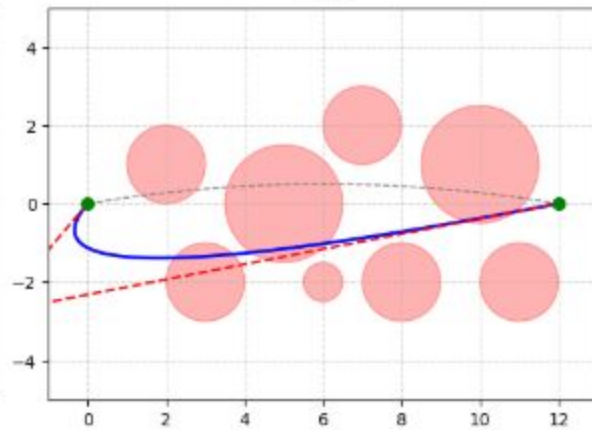
- Los gráficos que quieren mostrar los cambios de λ_{Obs} son los superiores. Están graficados con el $\lambda_{Suav} = 0.1$ que fue el dictado por la consigna 12)
- Los gráficos que quieren mostrar los cambios de λ_{Suav} son los superiores. Están graficados con el $\lambda_{Obs} = 10$ que fue el dictado por la consigna 12)

Grado $n = 2$

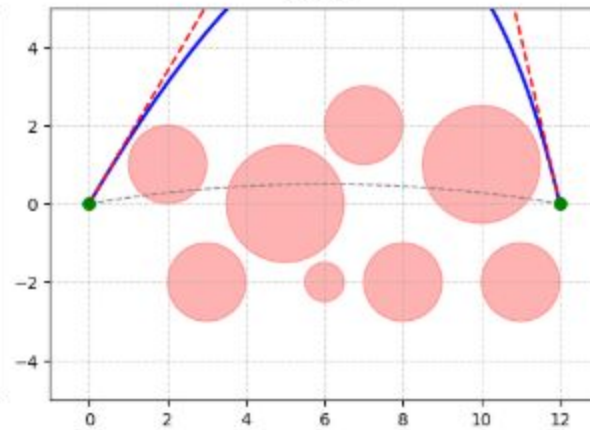
obs=1



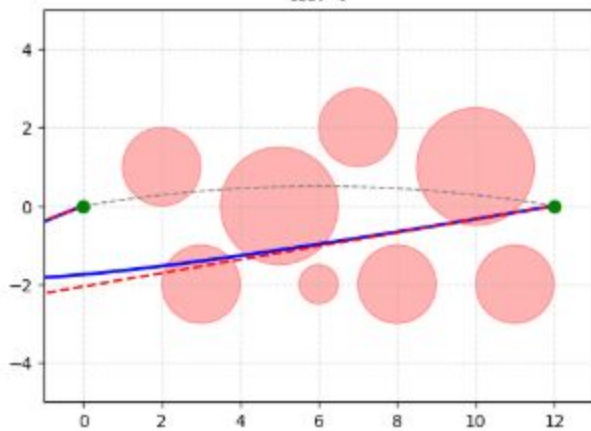
obs=10



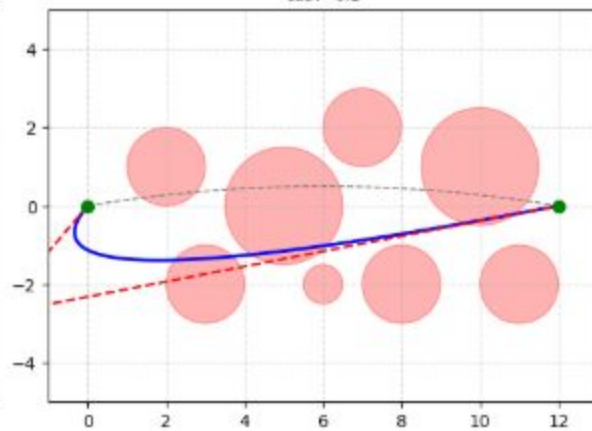
obs=100



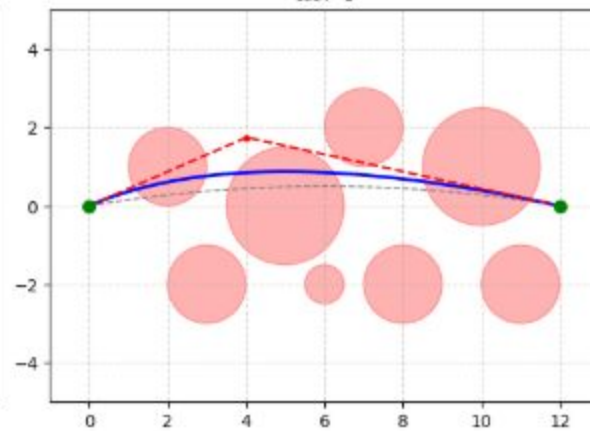
suav=0



suav=0.1

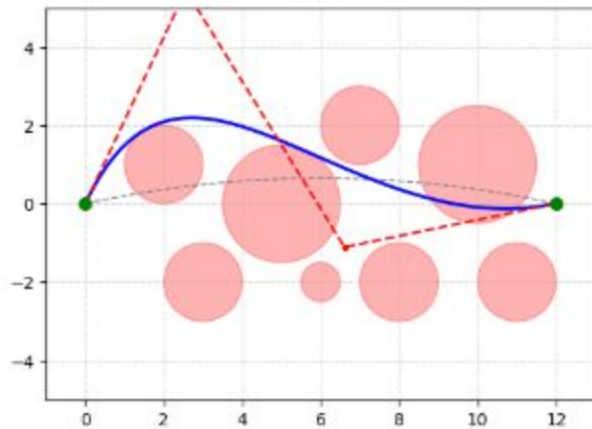


suav=1

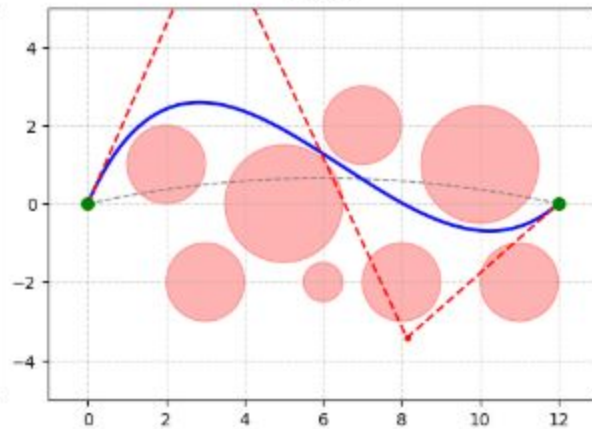


Grado $n = 3$

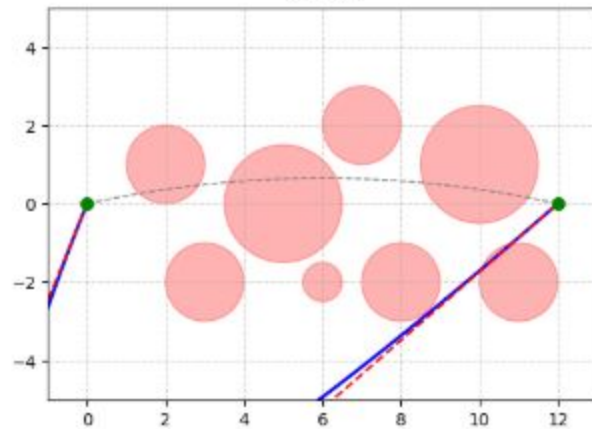
obs=1



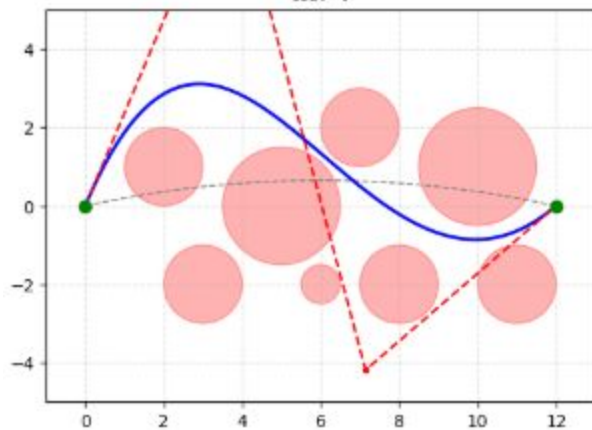
obs=10



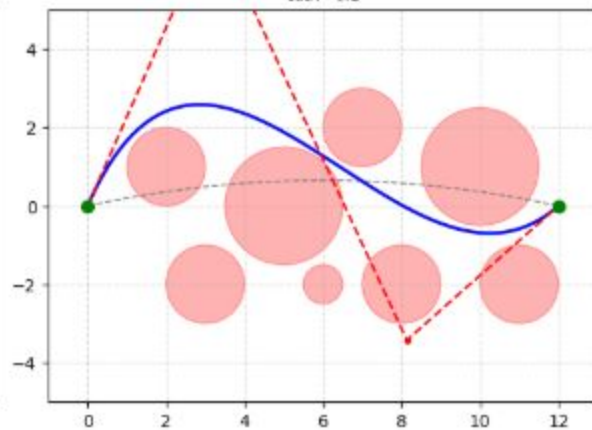
obs=100



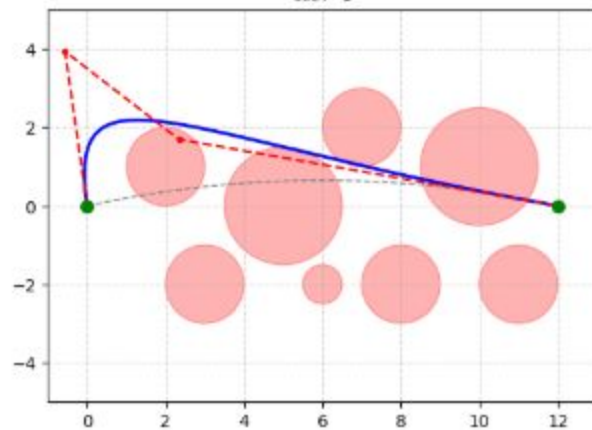
suav=0



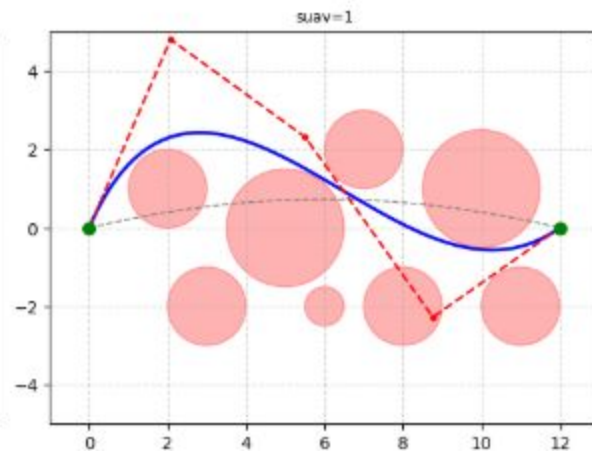
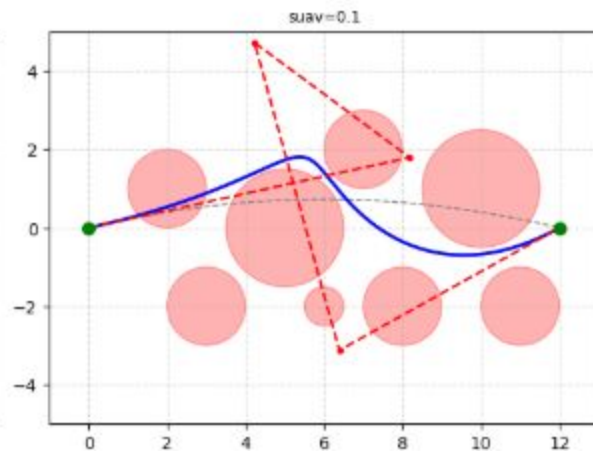
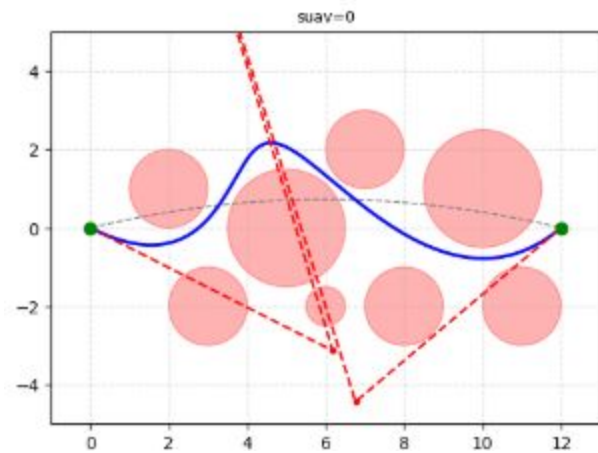
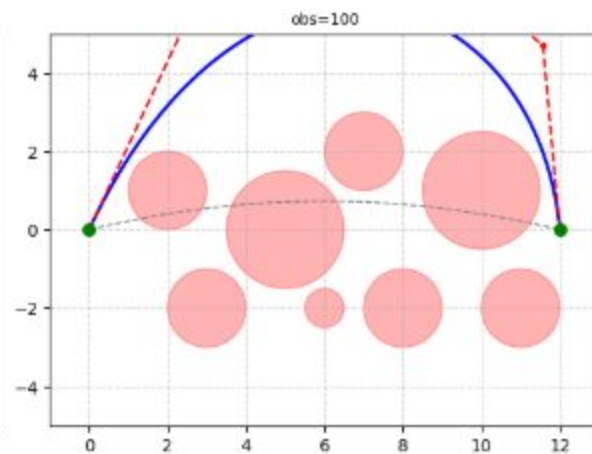
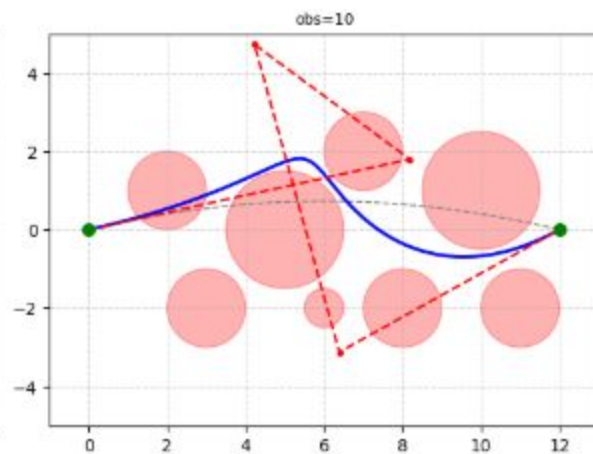
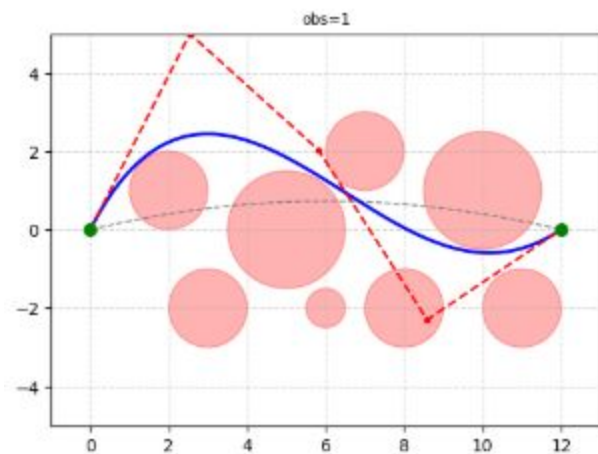
suav=0.1



suav=1

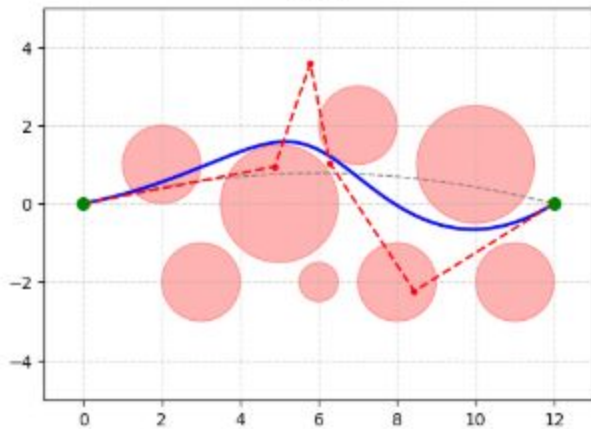


Grado $n = 4$

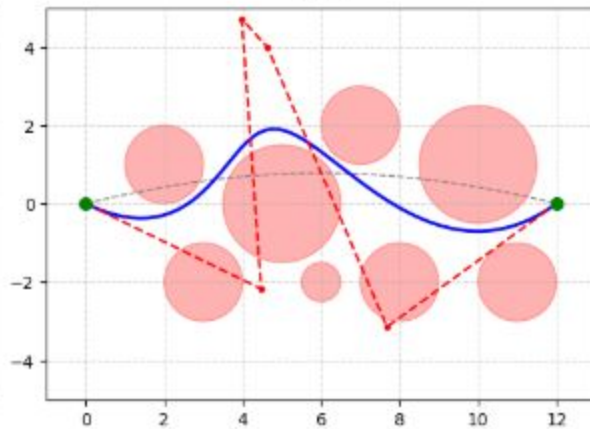


Grado n = 5

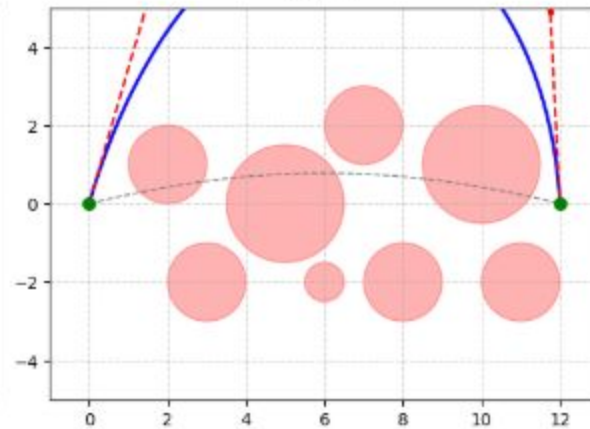
obs=1



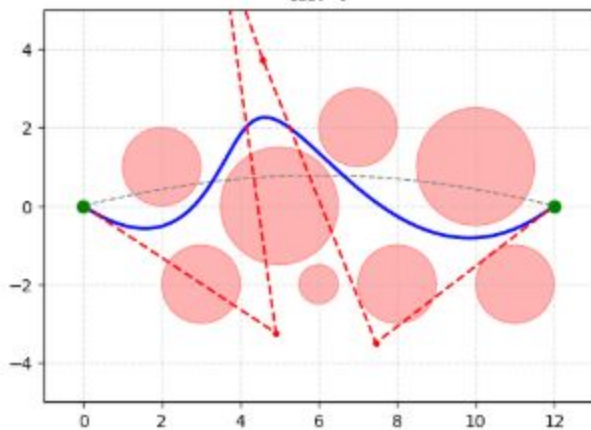
obs=10



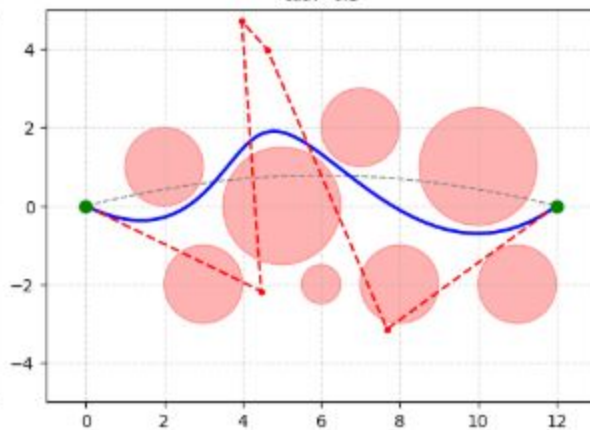
obs=100



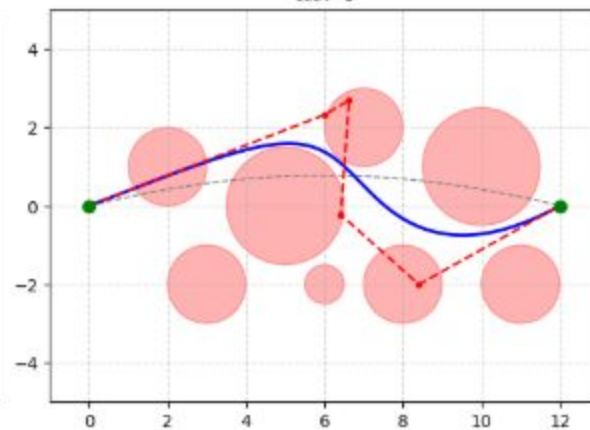
suav=0



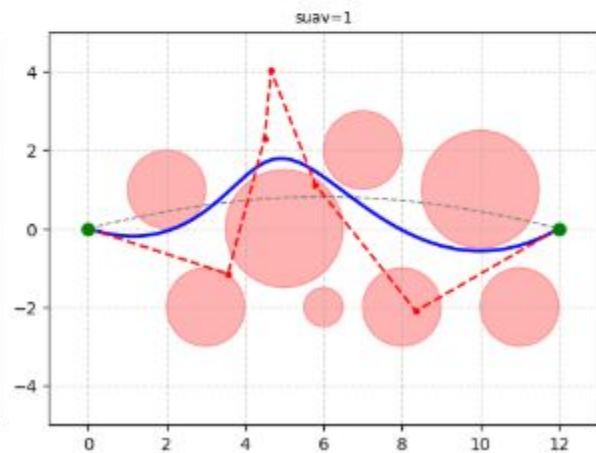
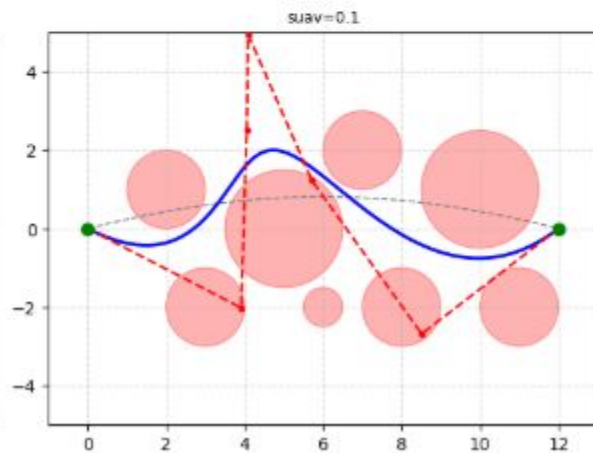
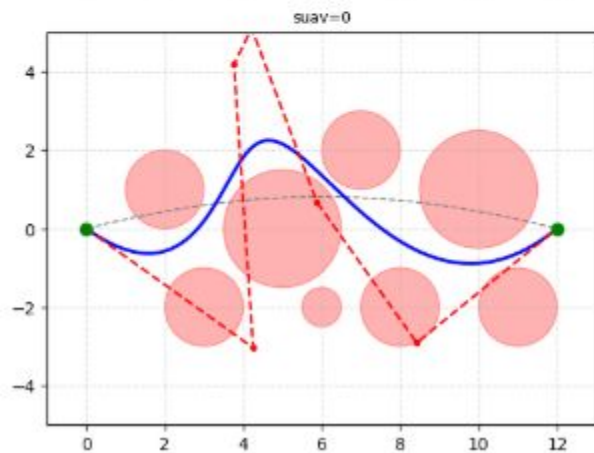
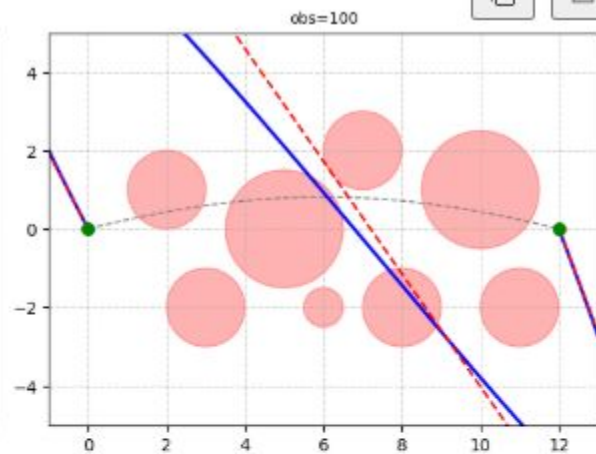
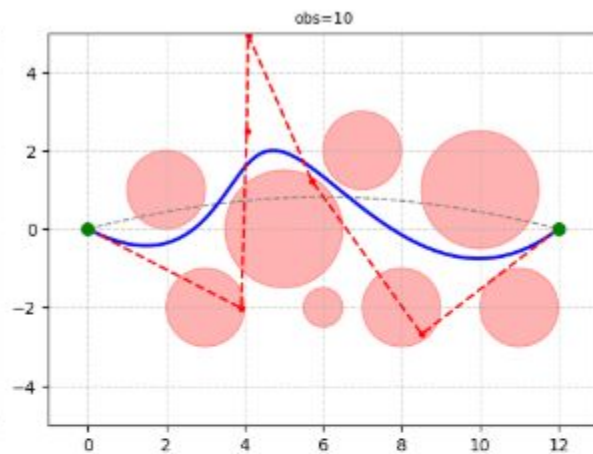
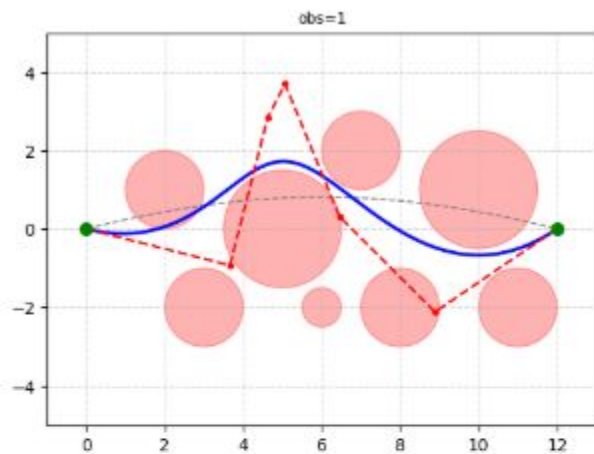
suav=0.1



suav=1



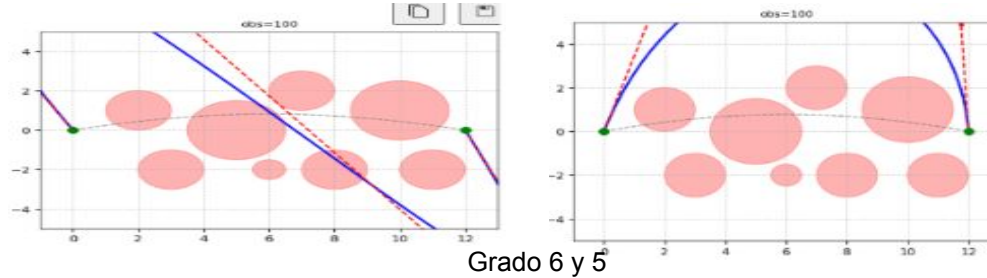
Grado $n = 6$



Cambios de lambda

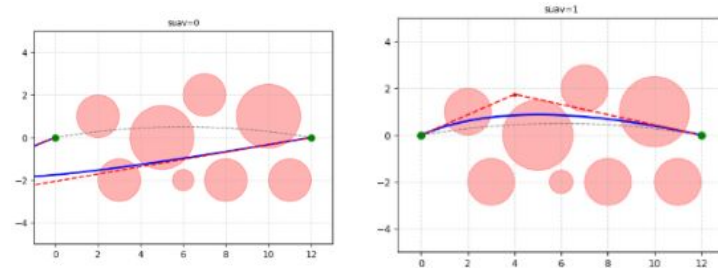
Reacción de las curvas ante aumento de lambdaObs: Al aumentar en cada gráfico los lambdaObs, vemos como cada vez más prioriza evitar los obstáculos.

Todos los casos donde se grafica con lambdaObs = 100 son curvas que prefieren hacer una curvatura enorme evitando la zona de los arbustos exageradamente.



Reacción ante lambdasSuav menores: Los gráficos con lambdaSuav = 0 nos permiten ver cómo las curvas buscan simplemente llegar al otro lado sin importarles el hacer ciertos cambios bruscos.

Pero, cuando las comparamos con los lambdasSuav= 1 vemos cómo corrigen esta tendencia y buscan una curvatura más suave. En el grado 2 es la muestra más notoria.



*Todo esto es debido a como está planteada la ecuación de error total en el ejercicio 12 (como los lambda acompañan a los errores en cada término)

Elección de trayectoria final

- *Elegimos entre las 5 del inciso 12
- *El criterio de factibilidad es distinto al planteado en el inciso 12) f)

Consideramos factibles a las curvas que no tienen colisiones.

Resumen de factibilidad y longitud por grado:

Grado	Colisiones	Factible?	Longitud	Dist. mín
2	3	False	13.5027	0.6536
3	0	True	14.2131	1.1549
4	1	False	13.3270	0.4478
5	0	True	13.7931	1.1433
6	0	True	14.0376	1.1446

Menor grado factible: $n = 3$

Comparación de longitudes entre grados factibles:

Grado 3: longitud = 14.2131

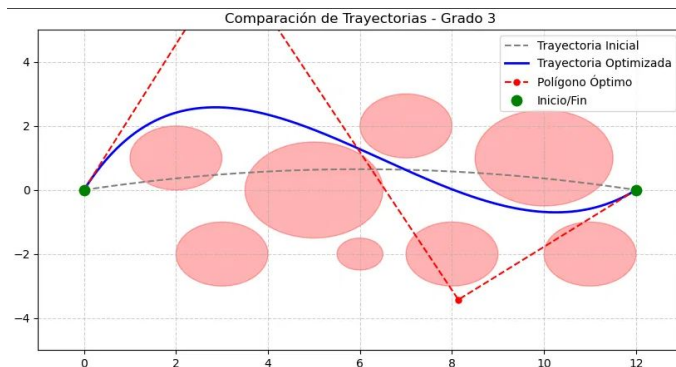
Grado 5: longitud = 13.7931

Grado 6: longitud = 14.0376

El grado con la trayectoria MÁS CORTA entre los factibles es: $n = 5$

Elección de trayectoria final

Elección final: Grado 3.



Longitud: Si bien no es la de menor longitud entre las factibles, la de grado 3 es solo un poco más larga que grado 5 (14.21 vs 13.79). No es una diferencia que cambie radicalmente el tiempo de poda o el gasto de batería de la podadora.

Distancia mínima a obstáculos: Tiene la mínima entre los 3. Eso es directamente un argumento de seguridad porque aunque no sea la más corta, es la que pasa más lejos de los arbustos en su punto más cercano. Para una podadora real, creemos que ese margen extra importa más que ahorrarse un poco de longitud.

Menor grado: 2 puntos de control intermedios contra 4 de grado 5. Menos parámetros significa una curva más fácil de describir, de mantener, con menos riesgo de zigzags raros y de no ser tan suave.

Párrafo de ideas de la materia

La función de costo total se construye sumando términos que tienen la forma de normas al cuadrado de diferencias, la misma estructura algebraica que vimos en el ajuste por mínimos cuadrados de los primeros ejercicios, solo que ahora aplicada a la longitud de la trayectoria y a la suavidad del polígono de control en lugar de al error entre una curva y datos observados.

También notamos que cada uno de estos términos es una forma cuadrática en las variables del problema, ya que se expresan como sumas de cuadrados de combinaciones lineales de los puntos de control.

En otras palabras, elevar al cuadrado una expresión lineal siempre termina generando productos cruzados entre las variables, que es justamente la definición de una forma cuadrática.

El término de suavidad cumple además el rol de una regularización, porque penaliza soluciones con curvatura brusca aunque estas minimicen mejor la longitud o evadan mejor los obstáculos, evitando así trayectorias poco realistas para un robot físico.

Como la función de costo combina estos términos con el de evitar obstáculos, que no es estrictamente cuadrático por el uso del máximo, ya no podemos resolver el problema de forma cerrada como en mínimos cuadrados clásicos, y necesitamos optimización numérica: usamos diferencias finitas para aproximar el gradiente y gradiente descendiente para ir actualizando los puntos de control de forma iterativa hasta encontrar una trayectoria que minimice el costo total.